

title: "Monitoring simultâneo 4G / 5G NR Cell / 5G NR DU Cell - Plan"
type: feat date: 2026-08-14 supersedes: docs/plans/2026-08-12-002-feat-5g-sa-nsa-monitoring-plan.md

Monitoring simultâneo 4G / 5G NR Cell / 5G NR DU Cell - Plan

0. O que a captura provou (e o que ela derrubou)

Este plano substitui o de 12/08, que assumia duas tasks PM chamadas **SA** e **NSA**. Os quatro HARs de `har-5g-oss/` mostram que essa premissa não existe no OSS. Tudo abaixo saiu de corpo de resposta real dos dois OSS, campo a campo — nenhuma linha é inferida.

0.1 A divisão real é por tipo de objeto, não por modo 5G

`GET /rest/oss/access/pm/v1/monitor/task/view-tree` enumera as tasks. As do evento de teste:

OSS	LTE	NR Cell	NR DU Cell
SP <code>10.220.50.9</code>	747 <code>Teste Santo Amaro</code>	749 <code>TesteSantoAmaro NRCELL</code>	748 <code>TesteSantoAmaro NRDUCELL</code>
OUTRAS <code>10.220.30.9</code>	2225 <code>TESTE FERRAMENTA - LTE</code>	2241 <code>TESTE FERRAMENTA - 5G CELL</code>	2242 <code>TESTE FERRAMENTA 5G DUCELL</code>

Não há nenhuma task "SA" nem "NSA" em nenhum dos dois OSS (98 tasks em OUTRAS, 106 em SP). O que o pedido original chamava de "SA" é a **NR Cell** (acessibilidade/queda/disponibilidade/usuários) e o que chamava de "NSA" é a **NR DU Cell** (PRB/throughput/volume/interferência).

0.2 Os dois conjuntos de contadores são disjuntos e já cobertos pelo catálogo atual

Cruzei `counterRes` real contra `core/kpi_formulas.py`. Interseção entre as duas tasks: **vazia**. Contadores da task não usados por nenhuma fórmula: **nenhum**, nos dois lados.

Task	Contadores	Métricas que fecham
NR Cell (749 / 2241)	20	<code>accessibility</code> , <code>drop_rate</code> , <code>availability</code> , <code>user_count</code> — 4/4, sem contador ausente
NR DU Cell (748 / 2242)	12	<code>utilization_dl</code> , <code>utilization_ul</code> , <code>throughput_ul</code> , <code>interference_ul</code> , <code>traffic_volume_dl_sa/_nsa</code> , <code>traffic_volume_ul_sa/_nsa</code> — 8/9

O catálogo 5G está certo como está. 12 das 13 definições calculam com os contadores reais. A reescrita proposta em 12/08 (dividir o catálogo à mão, apagar a subtração SA) era desnecessária e estava errada em 3 das 11 atribuições que propunha.

0.3 O único bloqueio real: `throughput_dl`

`throughput_dl` exige `N.ThpVol.DL`, `N.ThpVol.DL.LastSlot` e `N.ThpTime.DL.RmvLastSlot`. A task NR DU Cell traz o primeiro e o terceiro; `N.ThpVol.DL.LastSlot` não está configurado na task. É o único contador exigido por uma fórmula 5G e ausente das duas tasks.

Encaminhamento: pedir a inclusão do contador na task 748/2242 no iManager, não alterar a fórmula. Trocar a fórmula muda o significado da métrica para casar com uma configuração de task, que é justamente o acoplamento que este projeto já pagou caro. Até lá a métrica gera `invalid_formula` por ciclo — o mesmo comportamento já documentado e aceito para `ran_rtt` / `terrestrial_rtt` na task 2225.

0.4 A subtração SA não é gambiarra — é a única fonte do dado, e deve ficar

Nos 5 objetos da task 748, para DL e UL:

5G-SPSMG7-35-MB	N.ThpVol.DL = 1562879.456	N.NSA.ThpVol.DL = 1562879.456
5G-SPSMG7-35-MC	N.ThpVol.DL = 875625.352	N.NSA.ThpVol.DL = 875625.352
5G-SPSMH1-35-DA	N.ThpVol.DL = 36868.336	N.NSA.ThpVol.DL = 36868.336

Igualdade exata em 5/5 objetos, DL e UL. Ou seja: todo o tráfego é NSA e o volume SA é zero — e `traffic_volume_dl_sa = N.ThpVol.DL - N.NSA.ThpVol.DL` está medindo isso corretamente. Os dois contadores vivem na mesma task; a distinção SA↔NSA é de contador, nunca de task. Redefinir `traffic_volume_dl = N.NSA.ThpVol.DL` (proposta de 12/08) daria o mesmo número hoje e destruiria a capacidade de detectar tráfego SA quando ele existir. `_n_volume_dl_sa` / `_n_volume_ul_sa` ficam.

0.5 `objNo` é namespace da task, e em OUTRAS as tasks colidem

- Mesma célula, mesmo nome, **objNo diferente por task** (SP): `5G-SPSMG7-35-MB` = objNo `17714` na 749 e `17717` na 748.
- Em OUTRAS, 63 dos 105 objNo da task 2241 aparecem também na 2242 (ex.: `705`, `706`, `1089`, `1620` ...). Em SP os dois conjuntos são disjuntos.

Consequência no código atual, onde `self._obj_to_cell` é chaveado só por `obj_no` ([core/collector.py:1428](#)): a primeira task a resolver o objNo `705` grava o mapeamento, e a segunda task recebe esse mapeamento de volta pelo early-return `known = self._obj_to_cell.get(obj_no)` — **medição atribuída à célula errada, em silêncio**. Além disso `_request_objects_for_task` filtra o cache por tecnologia, então a segunda task deixa de pedir aquele objeto após a descoberta. Duas falhas silenciosas, do mesmo formato das já registradas em `ERRORS.md`.

A chave correta é `(task_id, obj_no)` — não `(technology, obj_no)` como propunha o plano anterior. `objNo` é alocado pela task; tecnologia é rótulo derivado. Só a chave por task resolve os dois casos acima, e resolveria mesmo se as duas tasks tivessem a mesma tecnologia.

Testar só em SP não teria pego isso: lá os objNo são disjuntos. É o mesmo padrão do objNo / objectNo — a diferença aparece numa regional só.

0.6 O que não muda entre regionais

- **Conjunto de contadores da NR Cell: idêntico nos dois OSS** (20 contadores, mesmos nomes).
- O regex atual `Cell Name\s*=\s*([^\s,]+)` já extrai o nome nos dois tipos de objeto: em `NR DU Cell Name=5G-SPSMH1-35-DA` ele casa o sufixo `Cell Name=...` e devolve o nome certo. **Nenhuma mudança de parser é necessária para DU Cell.**
- O nome da célula é **o mesmo** na NR Cell e na NR DU Cell — uma entrada de inventário serve às duas tasks. A resolução dinâmica por nome funciona; não é preciso `obj_no` manual no cadastro.
- `counterExpRes` vem `[]` nas duas tasks, nos dois OSS. Continua ignorável.

0.7 O que muda entre regionais (novo, para o registro)

- `objectNo / objectName` (OUTRAS, string) × `objNo / objName` (SP, int) — já tratado por `_obj_field`. Confirmado também nas tasks 5G.
- **Colisão de `objNo` entre tasks: existe em OUTRAS, não existe em SP (§0.5).**
- `POST monitor/task/<id>/start` devolve `objectList` com formas diferentes: OUTRAS usa `{fdn, objInstanceInfos}`, SP usa `{parentObj, childObjs, origObjCount...}`. **Não construir descoberta sobre `/start`** — o coletor descobre por `task/result` com `preExecTime: 0`, que é igual nos dois. Registrado para ninguém tentar.

0.8 Estado do inventário — o que dá para validar hoje

Evento	Células no inventário	Com 5G no nome	Casam com o que o OSS devolve
testesantoamaro	33	5	5 de 5 (5G-SPSMG7-35-MA/MB/MC , 5G-SPSMH1-35-DA/DB)
teste-curitiba	6.242	0	0 de 105

SP é validável ponta a ponta hoje, só adicionando as tasks 748 e 749 ao evento. OUTRAS não: o inventário de Curitiba não tem nenhuma célula 5G, então 100% dos 105 objetos da task 2241 caíram como `unmapped`. Isso é problema de dado/cadastro, não de código — ver Fase 0.

1. Decisões travadas por este plano

- **Três tecnologias:** `4G` , `5G_NRCELL` , `5G_NRDUCCELL` . São os nomes que o OSS usa nas tasks e o que separa dois conjuntos de contadores comprovadamente disjuntos. `SA / NSA` não aparecem em lugar nenhum do código — o que é SA e o que é NSA continua sendo distinguido por **contador**, dentro de `5G_NRDUCCELL` .
 - **Família para a interface:** `_TECH_FAMILY = {"4G": "4G", "5G_NRCELL": "5G", "5G_NRDUCCELL": "5G"}` . O operador vê `4G` e `5G` ; `NRCELL / NRDUCCELL` é chave interna de escopo de fórmula e nunca aparece na tela.
 - **Cache de objeto por** `(task_id, obj_no)` (§0.5).
 - **Tecnologia desconhecida (`None`) continua curinga para qualquer task.** É a correção de 13/08 que destravou Curitiba; a comparação por família não pode revogá-la.
 - **Nenhuma fórmula é apagada, nenhuma é reescrita.** Muda só o campo `technology` de 13 linhas do `CATALOG` , mais duas unidades que agora foram medidas.
 - **Zero custo de migração:** `kpi_measurements` tem `4G` em 100% das linhas nos dois bancos de produção (99.060 em Curitiba, 137.387 em Santo Amaro) e **nenhuma** linha `5G` . Nada a converter.
-

Fase 0 — Inventário 5G do evento de Curitiba (bloqueio de dado)

Sem células 5G no inventário, a task 2241 entrega 105 objetos e o coletor descarta todos. Antes de qualquer validação em OUTRAS, o inventário do evento precisa receber as células `5G-CT...` .

- Fonte pronta: os 105 `Cell Name=` da resposta da task 2241 (`har-5g-oss/har-monitoring-oss-tsl-5g-cell.har`).
- **Não é código.** É recadastro/importação no servidor central, pelo mesmo caminho que gerou as 6.242 células atuais.
- Verificação: `_log_unmapped` deixa de emitir WARNING para a task 2241, e o log do ciclo mostra `mapeados > 0` para ela.

Fase 1 e 2 não dependem desta fase. Só a validação em campo de OUTRAS depende.

Fase 2 — Coleta: três tecnologias no coletor

Explicação simples

Ensinar o coletor a rodar as três tasks no mesmo ciclo, cada uma com seu escopo de fórmulas e seu próprio espaço de `objNo` , sem que a segunda task 5G derrube o ciclo nem sequestre o mapeamento da primeira.

Escopo detalhado

`core/kpi_formulas.py` (mudança mínima, só rótulos e unidades medidas)

- Trocar `technology` de "5G" para "5G_NRCELL" em: `accessibility`, `drop_rate`, `user_count`, `availability`.
- Trocar para "5G_NRDUCCELL" em: `utilization_dl`, `utilization_ul`, `throughput_dl`, `throughput_ul`, `traffic_volume_dl_sa`, `traffic_volume_dl_nsa`, `traffic_volume_ul_sa`, `traffic_volume_ul_nsa`, `interference_ul`.
- Unidades agora medidas no `counterRes`: `N.ThpVol.DL / N.ThpVol.UL / N.NSA.ThpVol.*` vêm em `kbit`. Trocar "unidade OSS pendente" por "kbit" nas quatro `traffic_volume_*`.
- `production_ready`: manter `False` em `throughput_dl` (contador ausente, §0.3) e em `throughput_ul` — este último **calcula**, mas confirmar `Mbit/s` exige comparar um valor contra a tela do iManager, porque a unidade de `N.ThpTime.UE.UL.RmvSmallPkt` vem vazia na resposta. As quatro `traffic_volume_*` podem ir para `True`: são leitura direta/subtração de contadores cuja unidade foi medida.
- **Nada mais.** `_n_volume_dl_sa / _n_volume_ul_sa` ficam (§0.4).

`core/collector.py`

- `_normalize_task_technology(value)` — nova, separada de `_normalize_cell_technology`, usada só em `pm_tasks[].tech`: `4G / LTE → "4G"`; `NRCELL / NR_CELL / NR_CELL / 5G_NRCELL → "5G_NRCELL"`; `NRDUCELL / NR_DU_CELL / DUCELL / 5G_NRDUCELL → "5G_NRDUCELL"`. Valor não reconhecido: **logar em WARNING e descartar a task**. Hoje `_configured_pm_tasks` descarta em silêncio ([collector.py:335-337](#)) — um `tech` digitado errado no cadastro vira zero coleta sem nenhuma pista.
- `_TECH_FAMILY` — o mapa de §1.
- `_configured_pm_tasks`: passa a usar `_normalize_task_technology`. A trava de uma task por tecnologia continua, agora sobre os 3 rótulos. **Além disso: mover a chamada para dentro do try de `_collect_kpis_v2`** — hoje ela está fora ([collector.py:438](#)) e o `ValueError` sobe para o scheduler, matando o ciclo inteiro em vez de virar um `CollectionResult.error` diagnosticável.
- `_obj_to_cell` passa a ser chaveado por `(task_id, obj_no)`:
 - `_resolve_monitoring_cell(task_id, obj_no, obj_name, technology)` — recebe `task_id`; lookup e gravação usam a tupla.
 - Filtro de candidatos por **família**, preservando o curinga: `metadata.get("technology") is None or _TECH_FAMILY.get(metadata["technology"]) == _TECH_FAMILY[technology]`. A regra de candidato único (`len(choices) != 1`) fica intocada.
 - Mapeamento estático do `__init__` ([collector.py:755](#)): não há `task_id` conhecido ali. Gravar sob `(None, obj_no)` e fazer o lookup tentar `(task_id, obj_no)` e depois `(None, obj_no)`. (Na prática nenhum evento de campo tem `obj_no`, §0.6 — mas o caminho não pode ficar quebrado.)
 - `_request_objects_for_task(task)`: filtrar por `task_id` da chave, não por tecnologia. Fica mais simples e correto do que o filtro atual.
 - `_parse_monitoring_response`: passar `task_id` à resolução.
- **Cobertura**: `len(self._obj_to_cell)` deixa de ser o número de células mapeadas — com 3 tasks a mesma célula conta até 3 vezes e `cells_mapped` passaria de `cells_expected`, quebrando também o cálculo de `partial` ([collector.py:475](#) e [:481](#)). Trocar por `len({info["cell_id"] for info in self._obj_to_cell.values()})`.
- `MockCollector.collect_kpis` ([collector.py:2582](#)): hoje não marca `technology` em nenhuma linha. Passa a marcar; para células da família `5G`, gerar uma rodada `5G_NRCELL` (`accessibility, user_count`) e outra `5G_NRDUCELL` (`utilization_dl, throughput_ul, traffic_volume_*`), espelhando a divisão real de contadores. Necessário para validar a Fase 4 em `--mock`, sem VPN.

Código morto adjacente (só sinalizando, não mexer): `_legacy_collect_kpis` ([collector.py:1329](#)) e `_parse_kpi_response` ([:1630](#)) usam `_obj_to_cell` com a chave antiga ([:1342](#), [:1665](#)). Nenhum dos dois é chamado — `collect_kpis` retorna `_collect_kpis_v2` direto. Ficam inconsistentes com a nova chave. Registrar no commit; remoção é decisão à parte.

Testes

Fixtures entram pelo mesmo ponto que a resposta real entra — `_parse_monitoring_response` recebendo o corpo, nunca `_resolve_monitoring_cell` com argumentos já desembulhados (`ERRORS.md`, 14/08).

- Fixtures novas, extraídas dos HARs e sanitizadas: `tests/fixtures/pm_nrcell_sp_749.json`, `pm_nrducell_sp_748.json`, `pm_nrcell_outras_2241.json`.
- `test_as_tres_tasks_coletam_no_mesmo_ciclo` — 747/748/749 juntas, cada linha com a tecnologia da sua task.
- `test_objno_repetido_entre_tasks_nao_sequestra_a_celula` — **o teste central**: objNo 705 nas tasks 2241 e 2242 apontando para células diferentes; provar que a linha da 2242 não herda a célula resolvida pela 2241. Deve **falhar com a correção revertida** (disciplina de `ERRORS.md`).
- `test_celula_5g_resolve_nas_duas_tasks_5g` — célula cadastrada como 5G casa com `5G_NRCELL` e com `5G_NRDUCCELL` (prova da comparação por família).
- `test_celula_sem_tecnologia_continua_curinga` — regressão explícita da correção de 13/08, agora sob a comparação por família. Também deve falhar se alguém trocar o curinga por `_TECH_FAMILY[metadata["technology"]]`.
- `test_nrcell_fecha_as_quatro_metricas` / `test_nrducell_fecha_as_oito_metricas` — contra as fixtures reais, sem contador ausente além de `throughput_dl`.
- `test_throughput_dl_5g_reporta_contador_ausente` — trava o §0.3: se alguém “consertar” a fórmula removendo `N.ThpVol.DL.LastSlot`, este teste reprova.
- `test_volume_sa_e_a_diferenca_e_vale_zero_quando_todo_trafego_e_nsa` — trava o §0.4 contra a fixture real (`N.ThpVol.DL == N.NSA.ThpVol.DL` → `traffic_volume_dl_sa == 0`).
- `test_configured_pm_tasks_aceita_tres_e_rejeita_duas_iguais`.
- `test_tech_desconhecido_no_cadastro gera_warning`.
- `catalog_for_api()`: `5G_NRCELL` / `5G_NRDUCCELL` aparecem e `5G` não aparece mais.

Como validar

```
.venv\Scripts\python.exe -m pytest tests/test_kpi_monitoring.py tests/test_kpi_formulas.py -v
.venv\Scripts\python.exe -m pytest tests/ -q --basetemp=.pytest-work/tmp
```

Sem falhas novas além das 3 pré-existentis já documentadas em `MEMORY.md`.

Em campo (SP, hoje): adicionar `pm_tasks: [{tech:"4G",task_id:747},{tech:"NRCELL",task_id:749},{tech:"NRDUCCELL",task_id:748}]` ao `testesantoamaro` e rodar. Critério de saída, lido do log do ciclo: as três tasks aparecem em `tasks=`, `recebidos ≥ 15`, `nao_mapeados=0`, e `invalidos` correspondendo só a `throughput_dl` + `ran_rtt` / `terrestrial_rtt`.

Sugestão de commit

```
feat: collect 4G, 5G NR Cell and 5G NR DU Cell PM tasks in the same cycle
```

Fase 3 — Cadastro do evento: os três campos de task

Escopo detalhado

`server_frontend/index.html` (arquivo único, HTML+JS inline):

- Trocar `#pm-task-id` (:766) por três inputs numéricos opcionais: `#pm-task-4g`, `#pm-task-nrcell`, `#pm-task-nrducell`, rotulados "PM Task 4G / LTE", "PM Task 5G (NR Cell)", "PM Task 5G (NR DU Cell)", com o texto de ajuda citando o nome que a task tem no iManager (ex.: ... NRCELL (749)), que é como o operador a encontra.
- No submit (:1620): montar `integration.pm_tasks = [{tech, task_id}, ...]`, descartando vazios e não-numéricos.
- Em `editEvent` (:1730): popular a partir de `pm_tasks`; evento legado com só `pm_task_id` popula **apenas** o campo 4G, sem reescrever nada até salvar.
- `renderEvents` (:1556): trocar PM Task: X por 4G #747 · NR Cell #749 · NR DU Cell #748, omitindo as ausentes.
- `server.py`: sem mudança — `/api/events` grava o JSON recebido sem validar `integration`.

Dependência dura: esta fase não pode ir antes da Fase 2. Assim que o operador puder digitar duas tasks 5G, `_normalize_cell_technology` mapeia as duas para "5G" e o `ValueError` derruba o ciclo de KPI inteiro, 4G incluído.

Testes

Não há suíte cobrindo `server_frontend/index.html`. Validação manual (cadastrar com as 3 tasks → conferir card → reeditar → conferir pré-preenchimento → abrir evento legado → conferir que só o 4G vem preenchido → salvar → conferir `pm_tasks` no JSON gravado). Um Playwright mínimo no padrão de `tests/test_frontend_collection_ui.py` é desejável, não bloqueante.

Sugestão de commit

feat: add 4G/NR Cell/NR DU Cell task fields to event registration form

Fase 4 — Dashboard: agrupamento por família e filtro 4G × 5G

Escopo detalhado

`frontend/js/kpi.js`

- `_loadKpiCatalog` (:146) — **agrupar por família, não por tecnologia**. A regra atual joga em “Métricas comuns” toda métrica cujo `id` existe em mais de uma tecnologia (:162); com 3 tecnologias, `accessibility` (4G + NRCELL) e `throughput_ul` (4G + NRDUCELL) continuariam “comuns”, e um grupo por tecnologia exata ficaria quase vazio. Grupos finais: `Métricas comuns`, `Adicionais 4G`, `Adicionais 5G` — onde “comum” passa a significar *existe nas duas famílias*, e o rótulo de tecnologia da opção (`commonMetricTechnologies`, :185) mostra a **família** (4G/5G), nunca `5G_NRDUCELL`.
- `TECH_FAMILY` espelhando o backend, e `_cellTechFamily(cell)`.
- `_renderSiteList` / `_populateCellSelector`: aplicar `State.techFilter`. Célula sem família identificável permanece visível em qualquer filtro (falha aberta).

`api/api.py` — `get_sites` (:368) **não devolve células**, então `_renderSiteList` não tem hoje como saber a família de um site. Acrescentar ao dicionário de cada site um campo `tech_families: ["4G", "5G"]`, derivado **das linhas já persistidas** (`kpi_measurements.technology` do site) e, só como fallback, do inventário. Essa é a fonte certa: `MEMORY.md` (13/08) já travou que *a tecnologia do dado é a da task consultada, não o palpite pelo nome da célula*.

Por que não inferir do nome: em Curitiba as 6.242 células do inventário 4G se chamam `18NLCTAL01GI` — sem token de tecnologia. `resolveTechFreqFromId` (`server_frontend/index.html`) devolve `{tech:null, freq:null}` para elas. Portar essa heurística para o app criaria uma terceira implementação divergente e um filtro que não filtra nada naquela regional.

`frontend/js/state.js`: campo `techFilter` (`"all" | "4G" | "5G"`, default `"all"`).

`frontend/index.html`: bloco `#tech-tabs` acima de `#site-list`, no padrão de `#time-tabs`.

`frontend/css/main.css`: `.tech-tab` reaproveitando `.time-tab / .active`.

Testes

`tests/test_frontend_collection_ui.py` (mesmo padrão de servidor temporário + Chromium headless): o seletor expõe `Adicionais 5G` com entradas; a aba “4G” esconde sites sem célula 4G; a aba “5G” espelha; “Todas” restaura. Estender os cenários de `frontend/js/bridge.js` com sites de composição mista determinística — e atualizar o catálogo mockado de `bridge.js` (:173-178), que hoje ainda devolve `technology: "5G"`.

Sugestão de commit

feat: group metrics by technology family and filter sites by 4G/5G

Riscos e pontos em aberto

- **A task NR DU Cell de OUTRAS (2242) não teve corpo capturado.** O HAR traz as 3 requisições com status 200 e ~284 KB cada, mas sem `content.text`. Sei o `task_id` e os `objNo` pedidos, **não os contadores**. A NR Cell é byte a byte igual entre os dois OSS, o que torna razoável esperar o mesmo da DU Cell — mas *razoável não é medido*, e as duas falhas caras deste projeto vieram exatamente daí. O desenho acima não depende disso: contador ausente vira `invalid_formula` visível, não linha errada. **Recapturar com “Preserve log” + corpo de resposta antes de fechar a validação em OUTRAS.**
- **Unidade de `throughput_u1`.** Calcula, mas `N.ThpTime.UE.UL.RmvSmallPkt` vem sem unidade. Uma comparação contra a tela do iManager fecha o `production_ready`.
- **Volume de diagnósticos.** `invalid_formula` é emitido por objeto, por janela, por métrica, e `CollectionResult.diagnostics` não tem limite. Com 105 objetos × ~6 janelas acumuladas, um contador faltante gera milhares de entradas por ciclo. Já acontece hoje com `ran_rtt / terrestrial_rtt` na task 2225 ($116 \times 6 \times 2$). Não é regressão deste plano e está fora do escopo, mas deduplicar por `(metric, code)` antes de sair do parser é barato e vale um item separado.
- **Fase 0 é pré-requisito só da validação em OUTRAS, não do código.**